



MiT_b0_b0实验结果

BaseLine: CMX

网络调整:

- 对输入的rgb图像进行双线性插值以降低分辨率, 降低比例为原来的[0.5, 0.6, 0.7, 0.8, 0.9, 1.0];
- 在每个stage完成后的特征融合 (FRM+FFM) 之前使用双线性插值恢复rgb特征的分辨率, 并在融合之后使用双线性插值将rgb特征的分辨率降低至融合前的水平;
- 为了在将来使用不同backbone时仍然可以对齐 (FRM+FFM) 中rgb特征和depth特征的通道数, depth特征在特征融合前后均添加了一个 1×1 卷积;

实验细节:

- 使用NYUDepthV2数据集;
- 训练与评估策略均保持不变;

训练结果:

数据获取

- mIoU取500个epochs中的最高值;
- 参数量、浮点计算量与推理速度通过以下代码获取 (测速在单块3090显卡上进行) :

```

# Modified from: https://blog.csdn.net/Caesar6666/article/details/117926306
import os
import argparse
from importlib import import_module
import numpy as np
import torch
from torch.backends import import cudnn
import tqdm
from models.builder import EncoderDecoder as segmodel
import torch.nn as nn
from thop import profile

parser = argparse.ArgumentParser()
parser.add_argument("--config", help="train config file path")
parser.add_argument("--pth_path", help="train pth file path")
args = parser.parse_args()
# config network and criterion
cfg = getattr(import_module(args.config), "C")
if os.path.exists(cfg.log_dir):
    os.rmdir(cfg.log_dir)

print("#####")
print(f"Testing speed of {cfg.name}")
print("#####")

criterion = nn.CrossEntropyLoss(reduction="mean", ignore_index=cfg.background)
BatchNorm2d = nn.SyncBatchNorm
# model = segmodel(
#     cfg=cfg,
#     criterion=criterion,
#     norm_layer=BatchNorm2d,
#     syncbn=True,
# ).cuda()
model = segmodel(
    cfg=cfg,
    criterion=criterion,
    norm_layer=BatchNorm2d,
    syncbn=True,
)

```

```

state_dict = torch.load(args.pth_path)
state_dict = state_dict["model"]
model.load_state_dict(state_dict)
del state_dict
model.cuda()

cudnn.benchmark = True

device = "cuda:0"
repetitions = 300

dummy_input = (torch.rand(1, 3, 480, 640).cuda(), torch.rand(1, 3, 480, 640).cuda())
flops, params = profile(model, inputs=dummy_input)
print("the flops is {}G,the params is {}M".format(round(flops / (10**9), 2), round(params / (10**6), 2)))

# 预热, GPU 平时可能为了节能而处于休眠状态, 因此需要预热
print("warm up ...\n")
with torch.no_grad():
    for _ in range(100):
        _ = model(*dummy_input)

# synchronize 等待所有 GPU 任务处理完才返回 CPU 主线程
torch.cuda.synchronize()

# 设置用于测量时间的 cuda Event, 这是PyTorch 官方推荐的接口,理论上应该最靠谱
starter, ender = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
# 初始化一个时间容器
timings = np.zeros((repetitions, 1))

print("testing ...\n")
with torch.no_grad():
    for rep in tqdm.tqdm(range(repetitions)):
        starter.record()
        _ = model(*dummy_input)
        ender.record()
        torch.cuda.synchronize() # 等待GPU任务完成
        curr_time = starter.elapsed_time(ender) # 从 starter 到 ender 之间用时,单位为毫秒
        timings[rep] = curr_time

avg = timings.sum() / repetitions

```

```
print(f"\nAvg Latency={round(avg, 2)}ms\n")
FPS = 1000 / avg
print(f"Inference Speed: {round(FPS, 2)}FPS")
```

实验结果

分辨率降低比例	mIoU	Params(M)	Flops(G)	FPS
0.5	43.76	12.26	26.29	44.17
0.6	44.45	12.26	26.56	41.59
0.7	44.13	12.26	26.91	42.18
0.8	44.58	12.26	27.28	44.17
0.9	43.61	12.26	27.73	43.87
1.0	45.18	12.26	28.2	44.43

数据细节：FPS的测试均为各个网络测速10次后取得的最高值；

数据分析

1. 推测分辨率降低比例为[0.7, 0.9]的反常精度是训练过程本身的偶然因素影响；
2. MiT_b0_b0的主干使用分辨率为(480, 640)的数据集时，在3090显卡上，降低分辨率的行为**对于提高速度——精度权衡是无意义的**；

补充说明

关于测速细节

1. 使用以下操作会使同一个模型推理速度明显提高：

```
 cudnn.benchmark=True
```

2. 对比了对整个网络加载训练完成的权重和对主干加载预训练权重对推理速度测试的影响，发现几乎为0，因此考虑对于将要训练的MiT_b1_b0和MiT_b2_b0进行预测速，分析在这两种主干结构上降低分辨率操作的必要性；
3. 当前的测速脚本对同分辨率的同一个网络测速结果仍然有较大波动，但暂时想不到脚本的改

进空间;